



Banco de Dados para Tecnologia da Informação

UNIDADE 03

O novo normal

| MODELO LÓGICO

Na semana passada vimos que um projeto de banco de dados resumidamente utiliza quatro fases: a primeira trata da identificação do problema (ou seja: entender o que o cliente/empresa precisa resolver com o uso de banco de dados). Depois de entender o problema, parte-se para o segundo passo – a modelagem conceitual, que vimos na semana passada. Como você conseguiu perceber, é um passo puramente abstrato: não se trabalhou com código algum e muito menos com SQL. Foi realmente um passo para colocar as ideias (e requisitos) no papel de forma organizada com as entidades, atributos e relacionamentos.

Como uma analogia, vamos supor que você queira construir uma casa e tudo o que você possui até o momento é um terreno. A identificação do problema poderia ser comparada com uma reunião em um escritório de arquitetura e engenharia civil. Lá, arquitetos(as) e engenheiros(as) (análogos ao analista que trabalha com banco de dados) conversarão com você (o cliente) para entender as suas necessidades e entender o que você realmente de fato precisa no momento, quanto você pode gastar em tempo e dinheiro e, ainda, quanto tempo pretende fazer uso desta casa (afinal, se planeja durar por muitas e muitas décadas será preciso uma análise um pouco mais aprofundada quanto aos materiais que poderá usar).

Com as suas necessidades anotadas, estas pessoas traduzirão as suas necessidades para algo que eles dominam. Da mesma forma que uma empresa/cliente não entende muito de banco de dados, vamos supor que você também não entenda muito de engenharia civil e de arquitetura. É a responsabilidade destes profissionais criarem um projeto adequado (o que inclui espaço, ventilação, legislação municipal, hidráulica, elétrica, entre outros) e apresentarem isso de uma forma simples e de fácil entendimento, como uma lista resumida ou um orçamento prévio. Aqui é parecido com o diagrama ER que vimos na semana anterior. Ninguém colocou ainda a mão na massa. Nesta analogia, engenheiros(as) e arquitetos(as) ainda nem gastaram o seu tempo desenhando uma planta que, talvez, você nem aprovaria (ou iria querer pagar). Este passo serve para garantir que estão pensando na mesma coisa e que estejam conversando na mesma língua.

É depois disto que vem o terceiro passo: a modelagem lógica, o tópico inicial desta semana. Na analogia, a modelagem lógica seria como trabalhar efetivamente na planta da casa. Como a planta é um projeto no papel (ou no computador), ninguém consegue ainda morar dentro deste papel/tela do computador, mas já serve para entender como ela deve ficar no final das contas. Ela deve servir como uma ponte entre a

lista de requisitos do passo anterior (ou orçamento) e a casa finalizada. De nada adianta prometer uma coisa e entregar outra totalmente fora do combinado, não é mesmo?

Finalmente, ocorre o quarto e último passo: a modelagem física – análoga à construção da casa. Logo veremos mais sobre isso. Por enquanto, vamos nos concentrar na modelagem lógica. A conversão de um modelo relacional para um modelo lógico se dá pela transformação daquele diagrama ER em uma notação que dá mais detalhes sobre como as tabelas, colunas e seus relacionamentos devem se comportar.

Na modelagem lógica deixam de existir entidades, atributos e relacionamentos. Agora, nós temos:

1. **Tabelas:** um conjunto não-ordenado e independente de linhas.

- a. Usando os exemplos da semana anterior, poderíamos ter uma tabela chamada “clientes” e outra chamada “fornecedores”. A tabela “clientes” poderia conter várias linhas, onde cada linha é um (e somente um) cliente. Da mesma forma, cada linha da tabela “fornecedor” contém várias informações referentes a um (e somente um) fornecedor.
- b. Uma forma de imaginar isso é com um cadastro físico com folhas de papel: você pode ter todas as informações (ou todos os campos) de um cliente armazenadas em uma folha de papel. Dentro daquela folha, você poderá ter várias informações possíveis dele, e somente dele. Neste caso, não teríamos informações deste cliente salvas em outras folhas, e nem dados de outros clientes na folha dele. Afinal, seria muito difícil organizar desse jeito, não é? Também não teríamos uma ordem específica entre as folhas, mas somente que uma folha seria independente da outra. Você pode adicionar e remover quantas folhas quiser.

2. **Colunas:** os campos (características) referentes a uma tabela.

- a. Cada tabela pode ter um conjunto específico de campos/características/colunas independente de outra. Ou seja: a tabela “cliente” pode ter somente quatro colunas, enquanto a tabela “fornecedor” pode ter vinte colunas, ou vice-versa.
- b. As colunas possuem nomes que descrevem o que ela significa. Ou seja: não temos uma “coluna1”, “col2” ou similares. É preferível que tenhamos nomes como “telefone”, “nome completo” ou “CEP”.
- c. As colunas devem armazenar valores que façam sentido para aquela coluna. Ou seja: esperamos salvar somente números de telefone dentro de uma coluna chamada “telefone”, e somente nomes em “nome completo”. Não faria sentido guardarmos telefones na coluna “nome completo”, e vice-versa.
- d. Os valores devem ser atômicos e monovalorados. Ou seja: a coluna “telefone” da tabela “fornecedor” deve guardar dentro dela somente um número de telefone, e não cinco números dentro de um único campo. Se realmente há a necessidade de se ter mais de um telefone pode ser preferível ter uma tabela separada só para armazenar vários números dos fornecedores, por exemplo. **Não é**

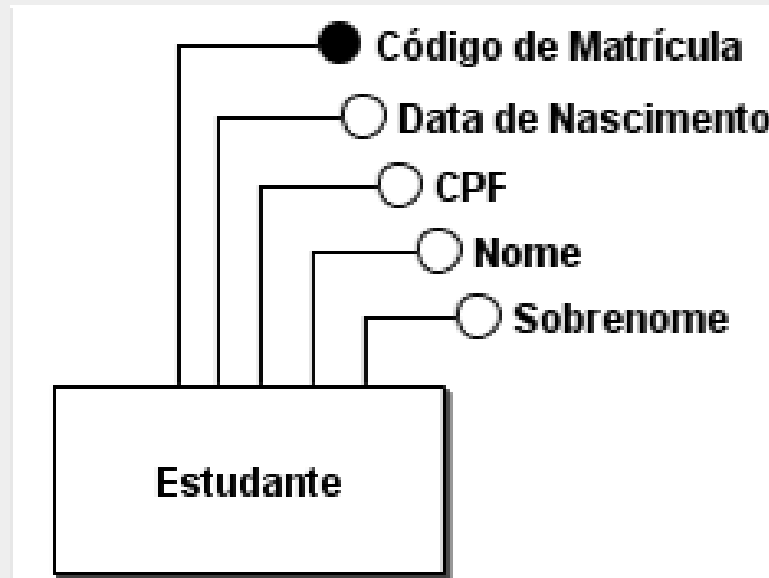
uma boa prática criar coluna como “telefone1”, “telefone2”, “telefone3” e assim sucessivamente por termos problemas de redundância (alguém colocar o mesmo telefone nas três colunas), dificultar as consultas (como fazer um cálculo da quantidade de números de telefone cadastrados dentre várias colunas) e limites (se temos 3 colunas de telefone como poderíamos tratar os clientes que queiram informar um quarto número por algum motivo?).

3. **Chaves:** são colunas identificadoras que fazem com que uma linha (indivíduo) seja completamente diferente dos outros. Você já deve ter visto coisas como “código do cliente”, “número do produto”, “número da nota fiscal” e similares enquanto esteve frequentando algumas lojas, não é? Estes identificadores são as *chaves primárias*.

a. Também temos as *chaves estrangeiras*: chaves primárias de outras tabelas que são usadas como referência por uma tabela. Uma tabela chamada “pedidos”, por exemplo, deveria conter informações como data e hora do pedido. Também poderia ter uma chave primária (identificador) que seria um número único atribuído automaticamente a cada novo pedido, certo? Contudo, também precisaríamos associar o pedido ao cliente de alguma forma. Não precisamos de *todos* os dados do cliente, mas somente o código dele para sabermos quem é o responsável por aquele pedido. Neste exemplo, uma coluna chamada “código do cliente” dentro desta tabela de pedidos é uma chave estrangeira.

A partir desta introdução, leia o capítulo **2. Modelo entidade-relacionamento** (páginas 34 a 47) do livro Pichetti, Roni Francisco et al.  **Banco de Dados** . Porto Alegre: SAGAH, 2020, para compreender melhor os conceitos de entidades, atributos, relacionamentos, etc.). Os conceitos de chave primárias e estrangeiras são de grande importância na compreensão desta aula.

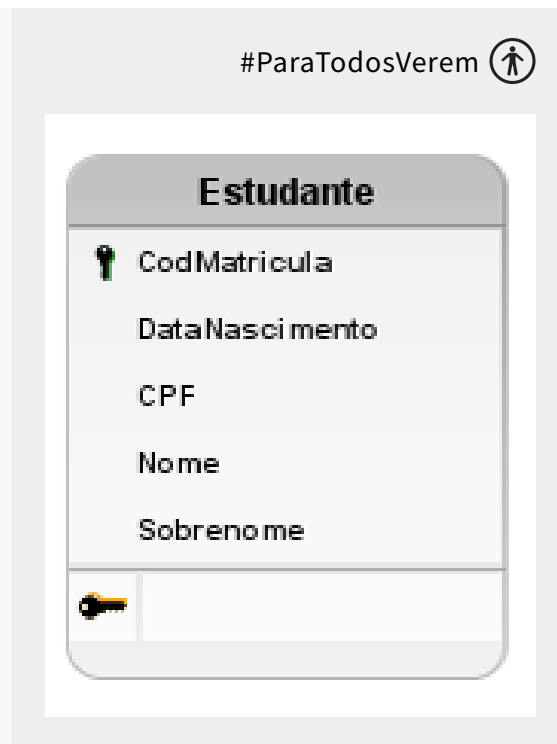
Assim, você verá que a representação do modelo conceitual deixa de ser assim:



Para se transformar nesta representação no modelo lógico:

Estudante (CodMatricula, DataNascimento, CPF, Nome, Sobrenome)

Ou, com uma representação mais visual:



Note que os nomes das colunas deixaram de ter espaços e acentos. Note também que o código de matrícula está sublinhado, denotando-o como chave primária. Finalmente, você pode estar se perguntando o motivo de termos um código de matrícula como identificador sendo que já temos o CPF como um identificador. Os motivos podem ser vários, dependendo da empresa e do sistema:

1. Nem todo estudante pode ter necessariamente um CPF no momento da matrícula, como um estudante de intercâmbio;
2. O CPF pode mudar no futuro, já o sistema da universidade pode ultrapassá-lo em sobrevivência (anos atrás, o CPF não existia. Em seu lugar, o Brasil tinha o CIC. Logo, o CPF é mais novo do que muitas universidades);
3. Uma única pessoa pode se matricular em momentos diferentes da vida na Universidade e, por isso, receber matrículas diferentes.

De forma resumida, um diagrama ER se converte em um modelo lógico com as seguintes transformações:

1. Entidades viram *tabelas*.
2. Cada atributo de uma entidade vira uma *coluna de uma tabela*.
3. Relacionamentos possuem diferentes tratamentos dependendo da sua cardinalidade:

- a. Relacionamentos muitos para muitos (também chamados de $n:n$) são transformados em uma nova tabela.
- b. Relacionamentos de 1 para muitos (também chamados de $1:n$) podem ser transformados pela adição de uma coluna em uma tabela para referenciar outra tabela.
- c. Relacionamentos de 1 para 1 (também chamados de $1:1$) podem ser transformados pela fusão de duas entidades em uma tabela só ou, ainda, ao adicionar uma coluna em uma tabela para referenciar outra.

Tabela 5.1 Alternativas para implementação de relacionamentos

Tipo de relacionamento	Regra de implementação		
	Tabela própria	Adição coluna	Fusão tabelas
Relacionamentos 1:1			
(0,1)  (0,1)	±	V	x
(0,1)  (1,1)	∓	±	V
(1,1)  (1,1)	∓	∓	V
Relacionamentos 1:n			
(0,1)  (0,n)	±	V	x
(0,1)  (1,n)	±	V	x
(1,1)  (0,n)	∓	V	x
(1,1)  (1,n)	∓	V	x
Relacionamentos n:n			
(0,n)  (0,n)	V	x	x
(0,n)  (1,n)	V	x	x
(1,n)  (1,n)	V	x	x

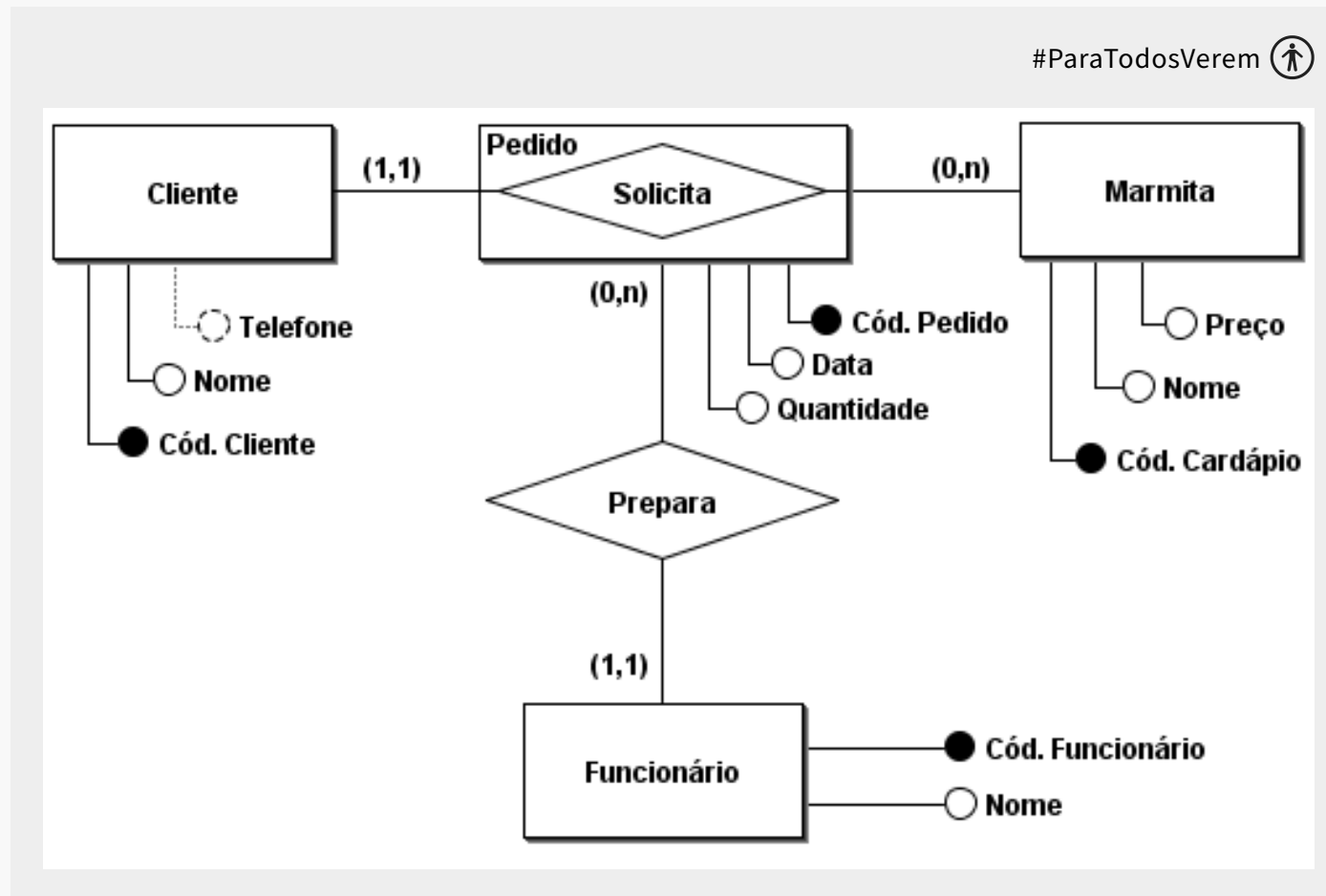
V: Alternativa preferida

±: Pode ser usada, primeira opção

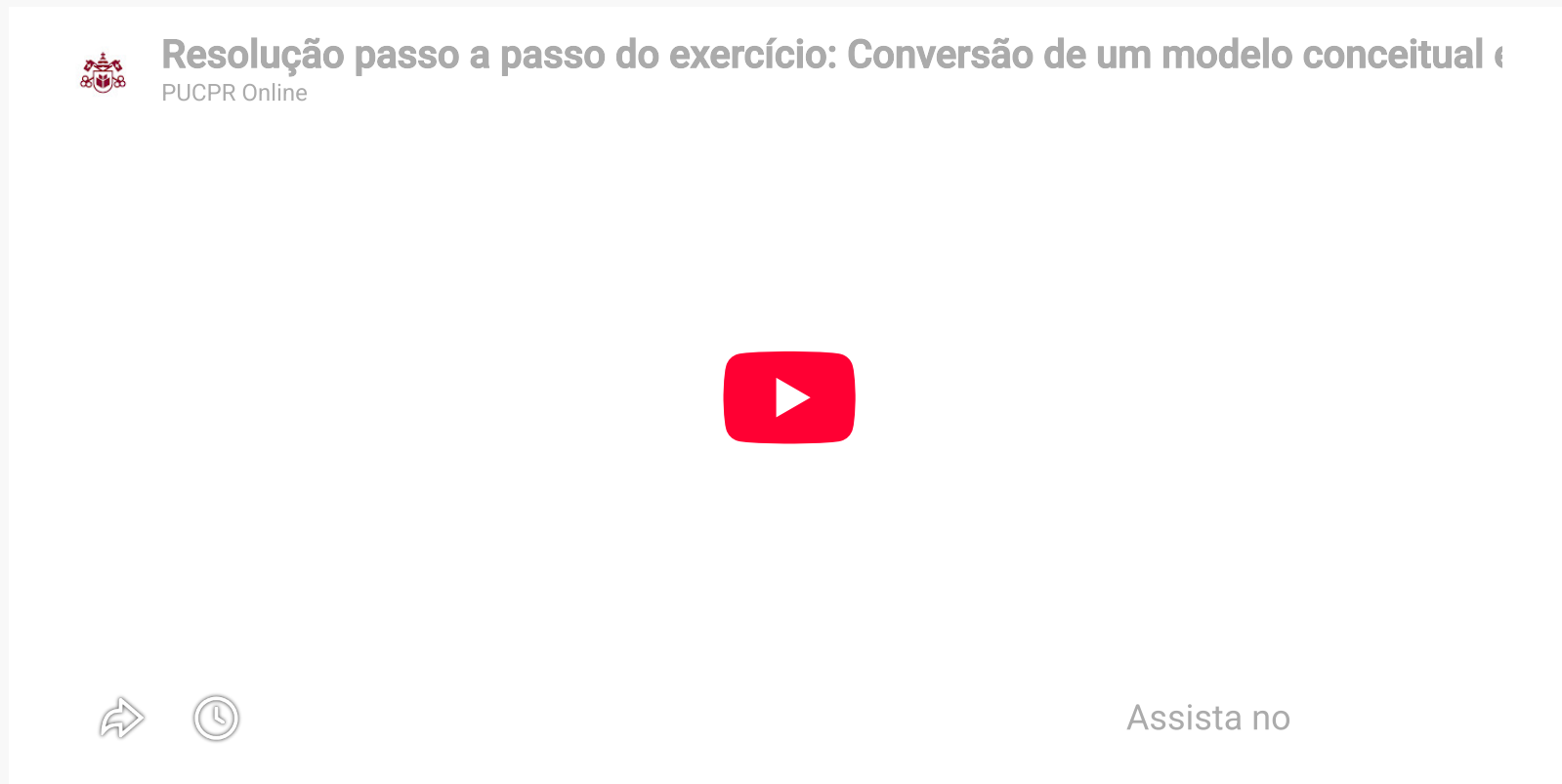
∓: Pode ser usada, segunda opção

x: Não cabe como solução

Desafio: vamos converter o modelo conceitual em lógico? Dado o modelo conceitual de uma marmitaria conforme imagem a seguir, crie o modelo lógico usando o MySQL Workbench.



| Vídeo Complementar - Resolução do exercício passo-a-passo.



| NORMALIZAÇÃO

O processo para finalmente termos as tabelas em um SGBD finaliza quando as tabelas são criadas junto com os seus relacionamentos usando chaves primárias e chaves estrangeiras. É, em bom português, quando todos os nossos CREATE TABLE funcionaram corretamente.

Por outro lado, um pouco antes de migrar do modelo lógico para o modelo físico é importante garantir que não temos redundâncias. Lembra que no início do conteúdo da semana comentamos sobre a importância da normalização? É um processo que garante que as nossas tabelas realmente não possuem problemas de redundâncias e inconsistências nos dados.

Por isso, leia mais sobre o processo de normalização no livro abaixo. Principalmente entenda as diferenças entre um banco não-normalizado (ÑN, ou NN) para a primeira, segunda e terceira forma normais (1FN, 2FN e 3FN, respectivamente).

Seção 4: **Normalização em banco de dados** (páginas 79 a 85) do livro Pichetti, Roni Francisco et al.  **Banco de Dados** . Porto Alegre: SAGAH, 2020.

Desafio: vamos verificar a normalização de nosso modelo lógico? Dado o modelo lógico feito no desafio anterior, realize a normalização até a 3FN.

| Vídeo Complementar - Resolução do exercício passo-a-passo.



Resolução passo a passo do exercício: Normalização

PUCPR Online



Assista no

| MODELO FÍSICO

Logo após a finalização do modelo lógico vem o modelo físico. O modelo lógico que foi produzido a partir do brModelo ou que está visível como um diagrama no MySQL Workbench não é ainda um banco de dados: é tão somente um projeto dele – é análogo à planta de uma casa. Agora, o modelo físico é análogo à casa construída.

No modelo físico será possível criarmos todas as estruturas das tabelas e deixá-las prontas para que outros sistemas possam se conectar e cadastrar dados. Pensemos em um aplicativo de celular para pedir comida: os clientes geralmente acessarão o aplicativo para fazer os seus pedidos, mas nunca terão acesso direto ao banco de dados. Por outro lado, o servidor conectado ao aplicativo terá um acesso direto e exclusivo

ao banco de dados por um “usuário de sistema” – um usuário e senha do banco de dados que foi criado somente para que este servidor possa ler e cadastrar informações. Isto é muito útil por uma questão de infraestrutura e de segurança da informação: é muito importante buscar proteger o banco de dados de acessos não-autorizados.

Dito isso, o modelo físico se dá ao usar uma linguagem estruturada de consultas. Não é *linguagem de programação*: o próprio nome diz que é uma *linguagem de consultas*. E, em inglês, isto se chama *structured query language*, ou SQL. Logo, soluções que usam e implementam o SQL seguem um padrão de linguagem (ainda que implementem algumas coisas de formas diferentes). É por isso que temos soluções como MySQL, SQL Server, PostgreSQL e demais soluções que vimos na Semana 1.

A criação do modelo físico se dá pela conversão da estrutura do modelo lógico em uma consulta (query) escrita usando a linguagem SQL. Praticamente toda query segue a seguinte estrutura:

[PALAVRA-CHAVE] [ONDE SE DEVE FAZER] [O QUE SE DEVE FAZER] [INFORMAÇÕES ADICIONAIS]

Aqui no modelo físico, o principal comando (palavra-chave) é para criar tabelas. Logo, em português seria mais ou menos assim:

CRIE UMA TABELA CHAMADA “Clientes” QUE TENHA AS COLUNAS DA SEGUINTE MANEIRA:
“CodCliente”, GUARDANDO SOMENTE NÚMEROS INTEIROS POSITIVOS E QUE ELES AUMENTEM SOZINHOS,
“Nome”, GUARDANDO LETRAS E NÚMEROS. É OBRIGATÓRIO E PODE GUARDAR ATÉ 80 CARACTERES,
“Telefone”, GUARDANDO LETRAS E NÚMEROS. É OPCIONAL E PODE GUARDAR ATÉ 11 DÍGITOS,
“CodCliente” SENDO A CHAVE PRIMÁRIA.

É claro que o SQL não entenderia a query acima. Por outro lado, em formato de SQL seria assim:

```
CREATE TABLE `Clientes` (  
  `CodCliente` INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  `Nome` VARCHAR(80) NOT NULL,
```

```
`Telefone` VARCHAR(11) NULL,  
PRIMARY KEY (`CodCliente`)  
)
```

Note que o campo de “telefone” aceitaria, no exemplo acima, letras e números. Ainda que isto seja indesejado, em alguns momentos a decisão de limitar ou não certos tipos de dados (como proibir que uma coluna de preços tenha valores negativos ou que uma coluna de telefone tenha letras) pode ocorrer a nível da aplicação e não do banco de dados. Esta geralmente é uma decisão de arquitetura de TI que envolve analistas responsáveis pela aplicação e pelo banco de dados.

O CREATE TABLE faz parte de um grupo de comandos específicos para a definição de dados no SQL (Data Definition Language, ou DDL). Todos os comandos do DDL ajudam na criação e manutenção do modelo físico. Desses, o CREATE TABLE é provavelmente o mais relevante. Existem outros como, por exemplo:

1. ALTER TABLE, para modificar tabelas já criadas (ex.: adicionar e remover colunas, alterar tipos de dados e configurações das colunas);
2. DROP TABLE, para remover tabelas já criadas;
3. TRUNCATE TABLE, para eliminar todos os dados (linhas) das tabelas.

Também é possível fazer isto em bases de dados/esquemas nas quais múltiplas tabelas existem e são agrupadas (exemplo: CREATE SCHEMA, DROP SCHEMA) e, ainda, em outros objetos de um banco de dados que não são cobertos na disciplina como views, triggers e índices.

Além disso, a leitura abaixo inclui exemplos adicionais do uso do CREATE TABLE em um modelo lógico de uma escola.

Seção 3: **Modelo físico de banco de dados para MySQL** (páginas 69 a 71) do livro: Pichetti, Roni Francisco et al.  **Banco de Dados** . Porto Alegre: SAGAH, 2020.

Antes de partirmos para o CREATE TABLE é importante entendermos que configurações existem em uma tabela que já criamos em um modelo lógico. Vamos lá?

Desafio: vamos converter o modelo normalizado em físico? Dado o modelo normalizado feito no desafio anterior, crie o modelo físico utilizando a linguagem SQL no MySQL Workbench.

| Vídeo Complementar - Resolução do exercício passo-a-passo.

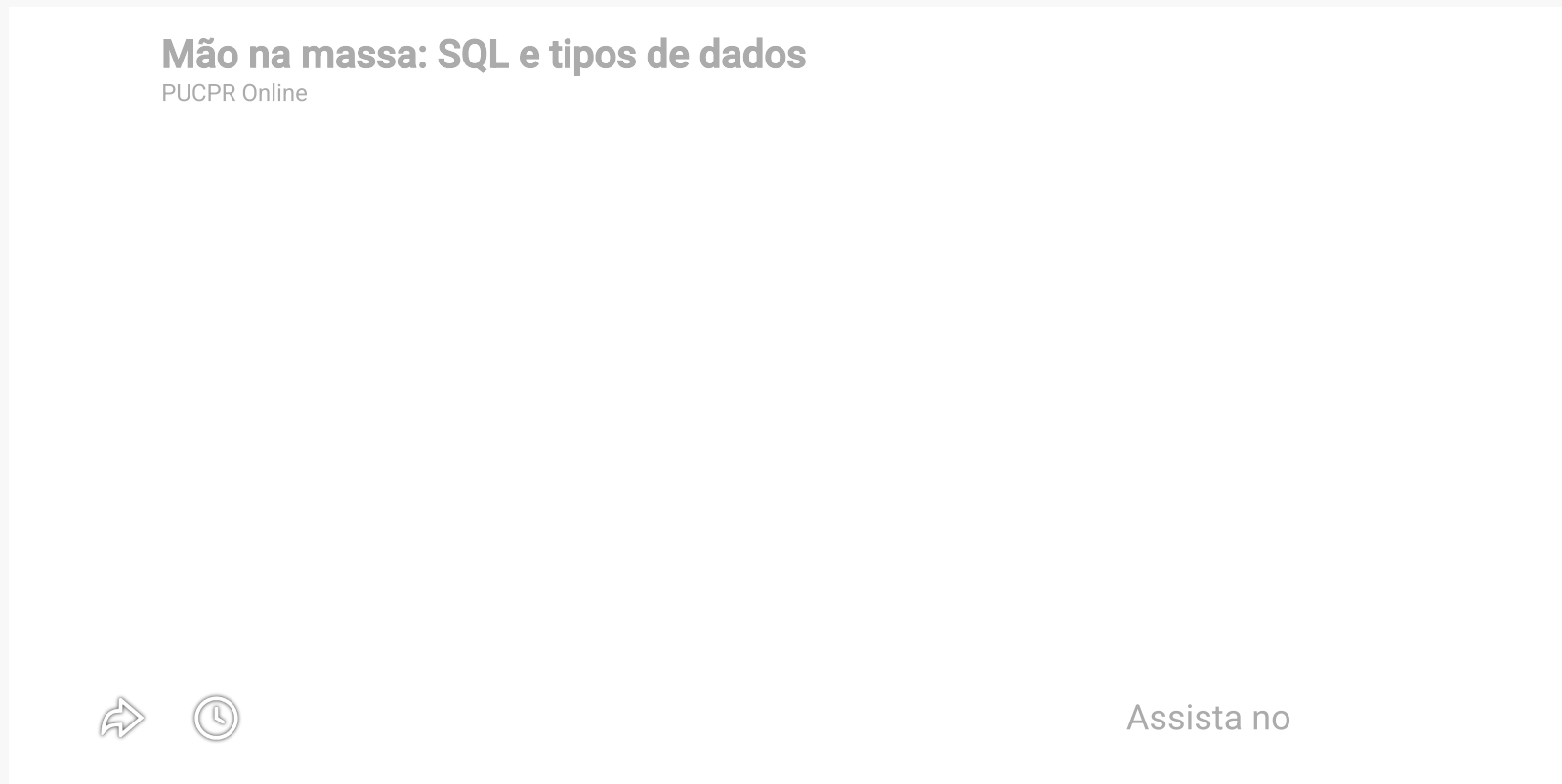
Resolução do exercício passo a passo

PUCPR Online



Assista no

| Mão na massa: SQL e tipos de dados



| CONCLUSÃO

Nesta semana vimos o processo de criação do banco de dados, indo do modelo conceitual, passando pelo lógico (e depois normalizando ele), para finalmente realizarmos o modelo físico usando a linguagem SQL. Na próxima semana iremos continuar o aprendizado do SQL, vendo como podemos interagir com o banco de dados.



© PUCPR - Todos os direitos reservados.